

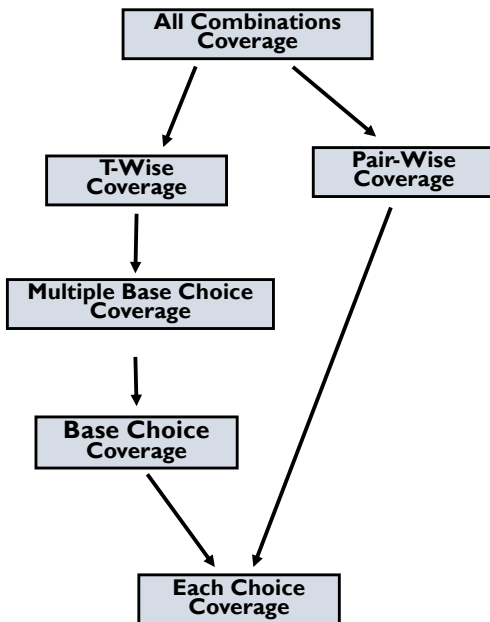
Q1

- (a) Draw a **control flow graph** for the function below on the opposite page. Make sure to list the basic block statements associated with each node and the predicates indicating different control flow. Number the nodes.

```
public static void function (int [ ] vals)
{
    int p = vals.length;
    double sum;
    int i=0;
    while (i < p)
    {
        sum += vals [i];
        i++;
    }
    if(p>0){
        System.out.println ("length: " + sum);
    }
    else{
        System.out.println("empty array");
    }
}
```

- (b) In the code, mark all **definitions** and **uses** of variable **p**
- (c) List two **def-clear paths** for variable **p** from the control flow graph

Q2. Is the following **subsumption relationship** for combination strategies **correct**? If not, please explain and fix.



Q3. (30 points) Use the following TSL of a small program to answer the following

```
Parameters:
  help:
    on.
    off.
  type:
    int.
    real.
    string.
Environments:
  file_contents:
    empty.  #[property Empty]
    non_empty.
  binary:
    yes.    #[if !Empty]
    no.
```

- (a) Assume you ignore all constraints (currently commented out) how many test frames does this TSL create? **(5 points)**
- (b) The **help** parameter (*on choice*) prints a help menu and exits the program immediately. Explain how this causes a problem for testing combinations in the frames created (give a concrete example of a test frame to explain).

(c) Rewrite the TSL using a constraint that will fix this issue

(d) Assume that the new constraint (from part c) is added and then uncomment the constraints represented by the [property Empty] and [if !Empty] annotations. How **many test frames are generated** ? Make sure to include the help parameter's new constraint. You can enumerate the actual frames if you are unsure

(e) **Drop the help parameter** from the model and **keep the constraints**. We now have three parameters/environments in the model (type, file_contents, binary). Assume a **base choice** of:

int, non-empty, yes

List the **complete base choice test** suite below. ***Do not include tests that violate the constraints. (5 points)***

(f) **List the pairwise test requirements for part e.** Make sure to consider constraints **(5 points)**

Q4: (20 points) Given the following Graph answer the questions below

$N = \{1, 2, 3, 4, 5\}$

$N_0 = \{1\}$

$N_f = \{5\}$

$E = \{(1, 2), (2, 2), (2, 3), (2, 4), (4, 3), (4, 5), (3, 5)\}$

(a) Draw the graph **(10 points)**

(b) List the test requirements for **Prime Path** Coverage **(5 points)**

(c) **Remove edge** $\{2,2\}$ from the graph. Does the following set of paths (together) satisfy *edge-pair coverage*? **Explain either way (5 points)**

$P1 = [1, 2, 4, 5]$

$P2 = [1, 2, 3, 5]$

Question 5. (10 points) Short Answers

(a) In t -way combinatorial coverage, t represents:

(b) Prime path coverage subsumes edge-pair coverage

True / False

(c) The TSL tool generates tests to satisfy the following coverage criteria (**select the strongest criteria it satisfies**)

1. All combinations coverage
2. Pair-wise coverage
3. Base choice coverage

(d) What is a **basic block** in a control flow graph? Give a small example.

There are two mutants in the findVal method, shaded and labeled MUTANT 1 and 2. For **EACH mutant** answer the following questions:

```
public class FindVal
{
    /**
     * Find last index of element
     *
     * @param numbers array to search
     * @param val value to look for
     * @return last index of val in numbers; -1 if absent
     * @throws NullPointerException if numbers is null
     */
    public static int findVal(int numbers[], int val)
    {
        int findVal = -1;

        for (int i=0; i<numbers.length; i++)
            // for (int i=(0+1); i<numbers.length; i++). - MUTANT 1
            if (numbers [i] == val)
                findVal = i;
            // findVal =0; - MUTANT 2
        return (findVal);
    }
}
```

- (a) If possible, find an input **that does not reach** the mutant(s)
- (b) If possible, find an input **that infects but does not propagate** the mutant(s)
- (c) If possible, find test inputs that **infects, propagates and kills** the mutant(s)
- (d) Define the mutation score of a test suite